Hybrid Pipeline for ESA Ground Station Scheduling
Classical Pruning + MWIS/QUBO + Quantum Annealing

Overview

The goal is to solve the ESA ground station scheduling and antenna allocation problem using a hybrid pipeline in which the instance is first reduced classically and only the hard reduced core is sent to a quantum annealer.

The main idea is:

> Start from the real scheduling problem, reduce it classically as much as possible, then send only the hard reduced conflict core to the annealer.

This follows the general logic of:

real scheduling problem → candidate contact slots → classical pruning → conflict graph → MWIS/QUBO

Key Rationale

A full direct formulation of the scheduling problem may be too large for current quantum annealing hardware. Therefore, the most realistic strategy is to use classical methods to reduce the search space first, and then apply quantum annealing only to a smaller conflict-dense subproblem.

1. Step 1: Exact Problem Scope

Before defining any graph, QUBO, or quantum annealing workflow, the first task is to specify precisely which version of the scheduling problem will be modeled. This step is essential because the overall ESA problem is broad and operationally rich, while the mathematical model used in the project must be concrete, finite, and internally consistent.

The goal of this first step is therefore to define:

- the planning horizon,
- the unit of decision,
- the relevant inputs,
- the first set of hard constraints,
- the objective function,
- and the initial modeling assumptions.

1.1. Planning Horizon

We consider a fixed scheduling horizon of 12 hours. All candidate communication opportunities, constraints, and decisions are defined only inside this interval.

Let

$$H = [T_0, T_0 + 12\text{h}]$$

denote the planning horizon, where T_0 is the start time of the scheduling instance.

This means that the model does not attempt to optimize over an indefinite or continuously changing future. Instead, it solves one finite planning instance over a fixed 12-hour window. This is important because it makes the problem tractable and consistent with a rolling-horizon interpretation, where a new instance could be generated later as time advances.

1.2. Decision Unit

The basic decision in the model is whether to select or not select a *candidate contact slot*.

A candidate contact slot corresponds to one full feasible visibility interval between:

- one satellite,
- one antenna,
- during one specific time interval.

Therefore, each decision is binary:

$$x_i = \begin{cases} 1 & \text{if candidate slot } i \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}$$

At this stage, we do not subdivide visibility windows into smaller pieces. Instead, one full feasible visibility interval is treated as one candidate slot. This is a deliberate simplification that reduces the number of decision variables and keeps the first model compact enough to later support a graph-based and QUBO-based formulation.

1.3. Inputs of the Model

For each candidate contact slot i, the model should include at least the following information:

- slot identifier,
- satellite identifier,
- antenna identifier,
- station identifier,
- start time,
- end time,
- duration,
- mission priority,
- urgency level,
- any optional operational metadata.

In addition to slot-level information, the model also requires system-level inputs:

- antenna availability windows,
- blackout windows,
- minimum and maximum allowed contact durations,
- required antenna reconfiguration or slew time between successive contacts.

These inputs define the feasible decision space from which candidate slots will later be generated and filtered.

1.4. Hard Constraints in the First Model

For the initial formulation, only the hard constraints that can be handled cleanly through preprocessing or pairwise conflicts are included. The first version of the model therefore uses the following hard constraints:

1. **Overlap conflict on the same antenna.** Two slots assigned to the same antenna cannot overlap in time.
2. **Reconfiguration-time conflict on the same antenna.** Two slots assigned to the same antenna cannot be scheduled too close in time if the required setup, slew, or reconfiguration time is not respected.
3. **Availability and blackout filtering.** Any slot that lies outside antenna availability or intersects a blackout interval is infeasible and must be removed before optimization.
4. **Invalid-duration filtering.** Any slot whose duration violates the allowed minimum or maximum duration is infeasible and must be removed before optimization.

These constraints are intentionally chosen because they fit naturally into the future conflict-graph representation. More complex global or higher-order constraints may exist in the real operational problem, but they are postponed to later validation stages.

1.5. Objective Function

The objective is to maximize the total operational value of the selected slots.

Each slot i is assigned a weight w_i based on three components:

- mission priority,
- contact duration,
- urgency.

The weight of slot i is defined as

$$w_i = \alpha\, p_i + \beta\, d_i + \gamma\, u_i$$

where:

- p_i is the priority score of slot i,
- d_i is the duration of slot i,

- u_i is the urgency score of slot i,
- $\alpha, \beta, \gamma > 0$ are tunable coefficients.

In practice, it is often useful to normalize these quantities so that they lie on comparable scales. A normalized version can be written as

$$w_i = \alpha\, \hat{p}_i + \beta\, \hat{d}_i + \gamma\, \hat{u}_i$$

with

$$\hat{p}_i, \hat{d}_i, \hat{u}_i \in [0, 1].$$

At this stage, the exact choice of the coefficients is not yet fixed; what matters is that the model already reflects the intended preference for higher-priority, longer, and more urgent contacts.

1.6. Initial Modeling Assumptions

To keep the first mathematical model consistent and suitable for a later MWIS/QUBO formulation, the following simplifying assumptions are adopted:

- Time is represented through precomputed candidate contact slots.
- Each slot is fixed in advance; the model does not optimize slot boundaries.
- A slot is either fully selected or fully rejected.
- Partial execution of a slot is not allowed in the first model.
- All relevant hard constraints are either enforced by preprocessing or represented later as pairwise conflicts.
- Higher-order, sequence-dependent, or global coordination constraints are not modeled in the first version unless they can be reduced to fixed pairwise incompatibilities.

These assumptions do not claim to describe the full operational richness of the real scheduling problem. Rather, they define a first compact abstraction that is mathematically clean and compatible with the conflict-graph approach needed for later quantum annealing experiments.

1.7. Result of Step 1

At the end of this step, the scheduling problem is defined as follows:

> Given a fixed 12-hour planning horizon, select a subset of candidate contact slots, where each slot represents one full feasible visibility interval between a satellite and an antenna, so as to maximize the total value of selected slots while respecting overlap, reconfiguration, availability, blackout, and duration feasibility constraints.

This completes the definition of the first mathematical problem scope and provides the basis for the next step: constructing the discretized set of candidate contact slots.

2. Step 2: Discretization Layer

Once the exact scope of the problem has been fixed, the next step is to transform the raw operational data into a finite set of candidate decisions. This is the purpose of the discretization layer.

In this project, the discretization step is particularly important because the later graph-based formulation requires a finite set of nodes. Therefore, before building any conflict graph, the model must first define which communication opportunities exist and which of them are feasible enough to be considered as candidate slots.

The output of this step is a weighted set of candidate contact slots that will later become the nodes of the conflict graph.

2.1. Input to the Discretization Layer

For each satellite–antenna pair within the 12-hour horizon, the discretization layer takes as input the raw visibility information and the operational metadata required to evaluate feasibility and value.

At minimum, the raw input should contain:

- satellite identifier,
- antenna identifier,
- station identifier,
- visibility start time,
- visibility end time,
- antenna availability windows,
- blackout windows,
- required reconfiguration or slew time,
- mission priority,
- urgency level,
- any additional optional metadata.

At this stage, the model still does not contain graph edges or optimization variables. The purpose here is only to determine which candidate contact slots exist.

2.2. Definition of a Candidate Contact Slot

In the chosen model, one candidate slot corresponds to one full feasible visibility interval. Therefore, a candidate slot s_i is defined as

$$s_i = (sat_i, ant_i, st_i, t_i^{start}, t_i^{end})$$

where:

- sat_i is the satellite associated with the slot,
- ant_i is the antenna associated with the slot,

- st_i is the station associated with that antenna,
- t_i^{start} is the start time of the feasible visibility interval,
- t_i^{end} is the end time of the feasible visibility interval.

The duration of slot i is then

$$d_i = t_i^{end} - t_i^{start}.$$

This definition means that each future graph node represents an entire possible communication opportunity, not a fraction of one. The main advantage of this choice is that it keeps the number of candidate nodes smaller and the later QUBO more compact. The main limitation is that the model cannot choose only part of a long visibility interval: it must either select the whole slot or reject it.

2.3. Feasibility Conditions During Discretization

Not every raw visibility interval becomes a valid candidate slot. Before inclusion in the candidate set, each interval must pass a basic feasibility test based on the hard constraints defined in Step 1.

A raw interval becomes a valid slot only if all the following conditions are satisfied.

(a) Horizon Consistency

The slot must belong to the 12-hour planning horizon:

$$[t_i^{start}, t_i^{end}] \subseteq H.$$

If a raw interval lies completely outside the horizon, it is discarded. If it only partially overlaps the horizon, one may either discard it or clip it to the horizon depending on the operational interpretation. In the first version of the model, the cleanest choice is to keep only those intervals that fit entirely inside the horizon.

(b) Antenna Availability

The slot must be contained in an antenna availability interval:

$$[t_i^{start}, t_i^{end}] \subseteq A_{ant_i},$$

where A_{ant_i} denotes the set of available time intervals for antenna ant_i.

If the antenna is unavailable during any part of the candidate interval, that interval is discarded.

(c) Blackout Compatibility

The slot must not intersect a blackout window:

$$[t_i^{start}, t_i^{end}] \cap B_{ant_i} = \emptyset,$$

where B_{ant_i} denotes the set of blackout intervals relevant to antenna ant_i.

This ensures that contacts are not considered during maintenance, interference avoidance periods, or any other forbidden operational interval.

(d) Valid Duration

The slot duration must satisfy:

$$d_{\min} \leq d_i \leq d_{\max},$$

where $d_{\min}$ and $d_{\max}$ are the minimum and maximum permitted contact durations.

Any slot violating this condition is discarded during preprocessing and never reaches the optimization stage.

2.4. Candidate Set Construction

After applying the previous feasibility rules, the surviving slots define the candidate set

$$S = \{s_1, s_2, \ldots, s_n\}.$$

Equivalently, one may define the candidate set as

$$S = \left\{ s_i \ : \ [t_i^{start}, t_i^{end}] \subseteq H,\ [t_i^{start}, t_i^{end}] \subseteq A_{ant_i},\ [t_i^{start}, t_i^{end}] \cap B_{ant_i} = \emptyset,\ d_{\min} \leq d_i \leq d_{\max} \right\}.$$

This set S is the finite decision universe from which the optimization problem will later select the final schedule.

2.5. Weight Assignment

Each feasible candidate slot $s_i \in S$ is assigned a value reflecting its operational importance. Following the modeling choices of Step 1, the weight of slot i is based on:

- mission priority,
- contact duration,
- urgency.

A first weight model is:

$$w_i = \alpha\, p_i + \beta\, d_i + \gamma\, u_i,$$

where:

- p_i is the priority of slot i,
- d_i is its duration,
- u_i is its urgency score,
- α, β, γ are positive coefficients.

If the three components are not naturally on the same scale, a normalized form is preferable:

$$w_i = \alpha\, \hat{p}_i + \beta\, \hat{d}_i + \gamma\, \hat{u}_i, \qquad \hat{p}_i, \hat{d}_i, \hat{u}_i \in [0, 1].$$

This weight is not yet used to resolve conflicts. Its role at the discretization stage is simply to enrich each candidate slot with the value information that the optimization model will later maximize.

2.6. Structured Representation of Each Slot

For implementation and mathematical clarity, each candidate slot should be stored together with all relevant attributes. A convenient representation is:

$$s_i = (sat_i,\, ant_i,\, st_i,\, t_i^{start},\, t_i^{end},\, d_i,\, p_i,\, u_i,\, w_i).$$

This makes it possible to treat the discretization output as a weighted candidate-slot list, where each element already contains the information required for:

- later pruning,
- conflict detection,
- graph-node creation,
- and objective-value evaluation.

2.7. Interpretation of the Discretization Choice

Choosing one full feasible visibility interval as one candidate slot has both benefits and limitations.

Advantages.

- It keeps the number of candidate nodes smaller.
- It reduces the future graph size.
- It makes the later MWIS/QUBO formulation more compact.
- It is easy to explain and easy to implement.

Limitations.

- It reduces flexibility.
- It prevents selecting only part of a long visibility interval.
- It may miss solutions that would require splitting one visibility window into several alternatives.

For a first mathematical model, this tradeoff is acceptable and even desirable, because it keeps the problem aligned with the project goal of producing a compact graph-based quantum-annealing formulation.

2.8. Result of Step 2

At the end of the discretization step, the model has produced a finite weighted candidate-slot set

$$S = \{(s_i, w_i)\}_{i=1}^{n},$$

where each s_i is one feasible full visibility interval and w_i is its associated value.

In words, the result of Step 2 can be summarized as follows:

From the raw visibility and operational data over the 12-hour horizon, construct a set of feasible candidate contact slots, where each slot represents one full feasible visibility interval between a satellite and an antenna, and assign each slot a weight based on priority, duration, and urgency.

This weighted set is the direct input to the next stage of the pipeline. In the following step, these candidate slots will be further reduced through classical pruning before being turned into nodes of the conflict graph.

3. Step 3: Classical Reduction Before Conflict-Graph Construction

After Step 2, the model provides a weighted candidate-slot set

$$S = \{(s_i, w_i)\}_{i=1}^{n},$$

where each s_i is a feasible candidate contact slot and w_i is its associated value. The purpose of Step 3 is to reduce this set before constructing the conflict graph.

This reduction stage is necessary because the later graph-based and QUBO-based formulations depend directly on the number of surviving candidate slots. In particular:

- each surviving slot will later become a node in the conflict graph,
- conflicts between surviving slots will become graph edges,
- and the size of the future QUBO will depend strongly on the number of surviving slots and conflicts.

Therefore, the role of Step 3 is not to solve the optimization problem yet, but rather to compress the candidate-space representation while preserving, as much as possible, the quality of the final solution.

The classical reduction stage is divided into three parts:

- **Step 3A: Constraint propagation**
- **Step 3B: Safe structural reductions**
- **Step 3C: Optional heuristic reductions**

This gives a concrete and implementable preprocessing pipeline that can be executed before graph construction.

3.1. General Structure of the Reduction Pipeline

The candidate-slot set is reduced in stages:

$$S \xrightarrow{\mathcal{P}_{CP}} S^{(A)} \xrightarrow{\mathcal{P}_{safe}} S^{(B)} \xrightarrow{\mathcal{P}_{heur}} S^{(C)},$$

where:

- $\mathcal{P}_{CP}$ is the constraint-propagation operator,
- $\mathcal{P}_{safe}$ is the safe structural reduction operator,
- $\mathcal{P}_{heur}$ is the optional heuristic reduction operator.

The resulting set $S^{(C)}$ is then used in the next stage to construct the conflict graph. If no heuristic reduction is applied, then the reduced set after Step 3 is simply $S^{(B)}$.

3.2. Purpose of the Reduction Stage

The reduction stage has three main purposes:

1. **Feasibility-driven filtering.** Remove candidate slots that are inconsistent with the operational constraints.
2. **Redundancy elimination.** Remove slots that are duplicates or strongly dominated by others.
3. **Instance-size control.** Optionally reduce the remaining set further when the instance is still too large for practical graph or QUBO construction.

This decomposition is important because it separates:

- mathematically justified reductions,
- and more aggressive reductions introduced only for scalability.

3.3. Step 3A: Constraint Propagation

The first part of the reduction stage consists of a deterministic filtering process based on the operational constraints already defined in Steps 1 and 2.

The goal of Step 3A is:

> Remove all candidate slots that are inconsistent with the modeled feasibility conditions before graph construction.

This is called constraint propagation because the model uses the known constraints of the scheduling problem to shrink the candidate-space representation before optimization.

3.3.1 Inputs Used in Step 3A

For each candidate slot s_i, the following information is already available from the previous steps:

- satellite identifier sat_i,
- antenna identifier ant_i,
- station identifier st_i,
- start time t_i^{start},
- end time t_i^{end},
- duration d_i,
- weight w_i,
- optional metadata such as priority and urgency.

In addition, the system-level operational constraints are known:

- planning horizon,
- antenna availability intervals,

- blackout windows,
- duration limits,
- reconfiguration or setup requirements.

These constraints are used to eliminate inconsistent candidate slots.

3.3.2 Step 3A.1: Horizon and Data Consistency

A slot must satisfy basic data consistency requirements. Remove any candidate slot with:

- invalid or missing timestamps,
- non-positive duration,
- missing satellite, antenna, or station identifier,
- inconsistent antenna–station mapping,
- start or end times outside the planning horizon.

In mathematical form, a slot must satisfy:

$$t_i^{start} < t_i^{end}, \qquad [t_i^{start}, t_i^{end}] \subseteq H.$$

Any slot that violates these conditions is removed.

3.3.3 Step 3A.2: Availability Consistency

A slot must lie completely inside the availability interval of its assigned antenna. Let A_{ant_i} denote the available time intervals for antenna ant_i. Then slot s_i is retained only if

$$[t_i^{start}, t_i^{end}] \subseteq A_{ant_i}.$$

Otherwise, the slot is removed.

3.3.4 Step 3A.3: Blackout Compatibility

A slot must not intersect a blackout interval. Let B_{ant_i} denote the blackout intervals associated with antenna ant_i. Then slot s_i is retained only if

$$[t_i^{start}, t_i^{end}] \cap B_{ant_i} = \emptyset.$$

Any slot intersecting a blackout interval is removed.

3.3.5 Step 3A.4: Duration Consistency

A slot must satisfy the minimum and maximum duration requirements:

$$d_{\min} \leq d_i \leq d_{\max}.$$

Any slot violating this inequality is removed.

3.3.6 Step 3A.5: Local Transition Consistency

The most important propagation step is the local transition consistency check.

Although the full conflict graph has not yet been built, the model already knows that reconfiguration or setup time between successive contacts may forbid some local timing patterns on the same antenna. Therefore, for each antenna, the candidate slots assigned to that antenna are inspected in temporal order.

Let R_{ant} denote the minimum required reconfiguration or separation time for antenna ant. For two slots s_i and s_j assigned to the same antenna, the forward transition from s_i to s_j is locally feasible only if

$$t_j^{start} - t_i^{end} \geq R_{ant}$$

whenever $t_i^{end} \leq t_j^{start}$.

The purpose of Step 3A.5 is not yet to build all pairwise conflicts, but to identify slots that are already locally inconsistent under the simplified timing logic. A slot may therefore be removed if, under the local transition rules of the chosen model, it cannot participate in any locally feasible scheduling pattern on its antenna.

This criterion is intentionally local and conservative. It provides a principled feasibility-based reduction before graph construction, without requiring a full combinatorial analysis at this stage.

3.3.7 Output of Step 3A

After applying all constraint-propagation filters, the result is a reduced set

$$S^{(A)} = \mathcal{P}_{CP}(S), \qquad S^{(A)} \subseteq S,$$

containing only candidate slots that are locally consistent with the modeled operational constraints.

Interpretation of Step 3A

Step 3A is not yet an optimization stage. Its purpose is purely to remove inconsistent candidate slots before graph construction. In other words, it shrinks the candidate space using feasibility logic, not value maximization.

3.4. Step 3B: Safe Structural Reductions

After constraint propagation, the next stage removes slots that are redundant in a structural sense. The goal of Step 3B is:

> Remove candidate slots that are duplicates or strongly dominated by others, without relying on aggressive heuristics.

These reductions are called *safe structural reductions* because they are designed to be as conservative and justifiable as possible under the first model.

3.4.1 Step 3B.1: Duplicate Removal

If two candidate slots represent exactly the same decision, then one of them is redundant and can be removed.

Two slots s_i and s_j are considered duplicates if:

$$sat_i = sat_j, \qquad ant_i = ant_j, \qquad st_i = st_j, \qquad t_i^{start} = t_j^{start}, \qquad t_i^{end} = t_j^{end}.$$

When this occurs, only one copy is retained.

This reduction is exact and should always be applied.

3.4.2 Step 3B.2: Strong Dominance Removal

The second safe reduction removes slots that are strongly dominated by others.

The intuition is the following: if one slot is at least as valuable as another and is no more restrictive under the current model, then the weaker slot can be removed without loss.

To make this implementable in the first version of the project, the comparison is restricted to slots inside the same local comparison class:

$$\mathcal{G}(r, a, s) = \{\, s_i \in S^{(A)} : sat_i = r,\ ant_i = a,\ st_i = s \,\}.$$

Within such a class, a slot s_j is strongly dominated by s_i if:

- $w_i \geq w_j$,
- $p_i \geq p_j$,
- $d_i \geq d_j$,
- $u_i \geq u_j$,

with at least one strict inequality.

Equivalently, s_j is removed if there exists a slot s_i in the same class such that

$$w_i \geq w_j, \qquad p_i \geq p_j, \qquad d_i \geq d_j, \qquad u_i \geq u_j,$$

and at least one of the above inequalities is strict.

This rule is intentionally strong and local. It avoids introducing ambiguous dominance comparisons between very different operational contexts while still removing clearly inferior alternatives.

3.4.3 Output of Step 3B

After duplicate removal and strong dominance filtering, the result is

$$S^{(B)} = \mathcal{P}_{safe}(S^{(A)}), \qquad S^{(B)} \subseteq S^{(A)}.$$

This set contains only locally consistent, non-duplicate, and non-dominated candidate slots under the chosen first-model assumptions.

3.5. Step 3C: Optional Heuristic Reductions

Even after constraint propagation and safe structural reductions, the instance may still be too large for practical graph construction, QUBO construction, or quantum annealing. In such cases, additional size-control reductions may be introduced.

The goal of Step 3C is:

Reduce the candidate-slot set further when necessary, even if this requires using heuristic rules that do not guarantee preservation of the exact optimum.

This stage is explicitly optional and must be benchmarked carefully.

3.5.1 Heuristic Nature of Step 3C

Unlike Steps 3A and 3B, this stage is not guaranteed to preserve the exact optimal solution of the reduced problem. Therefore:

- Step 3C should be used only when needed,
- the chosen heuristic must be stated explicitly,
- and its effect on solution quality should later be evaluated experimentally.

3.5.2 Step 3C.1: Top-k Reduction Per Satellite

The first and recommended heuristic is to keep only the top-k candidate slots for each satellite.

For each satellite r, define

$$S_r^{(B)} = \{\, s_i \in S^{(B)} : sat_i = r \,\}.$$

Then sort the elements of $S_r^{(B)}$ in descending order of weight and retain only the first k slots. The resulting reduced set is

$$S_r^{(C)} = \mathrm{Top}_k(S_r^{(B)}).$$

The global reduced set after this heuristic is then

$$S^{(C)} = \bigcup_r S_r^{(C)}.$$

This heuristic is attractive because:

- it is easy to implement,
- it is easy to explain,
- it preserves at least a controlled number of alternatives for each satellite,
- and it avoids allowing one satellite with many alternatives to dominate the entire candidate set.

For this reason, Top-k per satellite is the recommended first implementation of Step 3C.

3.5.3 Step 3C.2: Weight Percentile Filtering

A second possible heuristic is global percentile filtering. In this approach, the weights of all slots in $S^{(B)}$ are ranked, and only the top fraction is retained.

For example, if the bottom $q\%$ of weights is discarded, then

$$S^{(C)} = \{\, s_i \in S^{(B)} : w_i \geq \tau_q \,\},$$

where τ_q is the weight threshold corresponding to the chosen percentile.

This method is simple, but it is less balanced than Top-k per satellite because it may remove all candidate slots for some satellites if their weights are systematically lower than others.

Therefore, this heuristic is not recommended as the first implementation, but it remains a possible alternative.

3.5.4 Step 3C.3: Near-Duplicate Weak Alternative Reduction

A third possible heuristic is to group operationally similar slots and retain only a few representatives.

For example, one may group slots that share:

- the same satellite,
- the same station,
- nearly identical time intervals,
- and similar values.

Within each group, only the strongest few slots are kept, while weak near-duplicates are removed.

This heuristic is useful when the candidate set contains many almost-equivalent alternatives that inflate the future graph without adding significant decision diversity. However, it requires a more elaborate grouping definition and is therefore better suited to later extensions of the implementation rather than the first prototype.

3.5.5 Step 3C.4: Low-Value Congestion-Based Filtering

Another possible heuristic is to remove candidate slots that combine:

- low value,
- and membership in a highly congested local timing region.

This idea is appealing because it tries to remove weak candidates precisely where competition is highest. However, it requires defining congestion before the full conflict graph is constructed. For that reason, it is conceptually useful but less convenient as a first implementation.

3.5.6 Recommended Choice for Step 3C

Given the need for a specific and implementable first pipeline, the recommended heuristic for the initial prototype is:

$$\boxed{\text{Top-}k \text{ reduction per satellite}}$$

The other heuristic options should be treated as later variants to benchmark if needed.

3.6. Final Output of Step 3

The final output of the classical reduction stage is a reduced candidate-slot set:

$$S^\star = \begin{cases} S^{(B)} & \text{if no heuristic reduction is used,} \\ S^{(C)} & \text{if Step 3C is applied.} \end{cases}$$

This set $S^\star$ is the input to the next stage, where the conflict graph will be constructed.

In words, the result of Step 3 can be summarized as follows:

> Starting from the weighted candidate-slot set produced by the discretization layer, apply a classical reduction pipeline composed of constraint propagation, safe structural reductions, and optional heuristic filtering, in order to obtain a smaller candidate set suitable for conflict-graph construction and later QUBO generation.

3.7. What Step 3 Achieves in Practice

This stage should be evaluated not only conceptually, but also quantitatively. For each instance, it is useful to record:

- the number of candidate slots before reduction,
- the number after Step 3A,
- the number after Step 3B,
- the number after Step 3C, if used.

A natural reduction ratio is:

$$\rho = 1 - \frac{|S^\star|}{|S|}.$$

This quantity measures how much the candidate-slot space has been compressed before graph construction.

Final Interpretation of Step 3

Step 3 is a fully classical reduction stage applied before graph construction. It does not yet solve the scheduling problem, but it plays a critical role by shrinking the candidate space so that the later conflict graph and QUBO remain tractable enough for classical benchmarking and quantum annealing experiments.

4. Step 4: Conflict-Graph Construction

After Step 3, the model provides a reduced candidate-slot set

$$S^\star = \{(s_i, w_i)\}_{i \in I^\star},$$

where each surviving candidate slot is:

- feasible under the modeled operational constraints,
- non-duplicate,
- non-dominated under the chosen safe reductions,
- and, if Step 3C has been used, possibly further reduced by heuristic filtering.

The purpose of Step 4 is to transform this reduced set into a weighted conflict graph. This graph is the central object that connects the scheduling problem to the Maximum Weight Independent Set (MWIS) formulation and, later, to the QUBO representation required for quantum annealing.

In particular, the objective of this step is:

Construct an undirected weighted graph in which each node represents one surviving candidate contact slot and each edge represents a pairwise incompatibility between two slots.

4.1. Output of Step 4

The output of this stage is a weighted undirected graph

$$G = (V, E, W),$$

where:

- V is the set of vertices,
- E is the set of conflict edges,
- $W : V \to \mathbb{R}_{>0}$ is the node-weight function.

In the present model:

- each vertex corresponds to one surviving candidate slot,
- each edge indicates that the two associated slots cannot both be selected in the same schedule,
- each node weight equals the operational value of the corresponding slot.

This weighted conflict graph is the mathematical structure that will later define the MWIS problem.

4.2. Node Definition

The first step of graph construction is to define the node set.

For every candidate slot $s_i \in S^\star$, create one graph node $v_i \in V$. Therefore, there is a one-to-one correspondence:

$$v_i \longleftrightarrow s_i.$$

The node weight is inherited directly from the slot weight:

$$W(v_i) = w_i.$$

In practice, even though the graph is represented mathematically as an abstract set of vertices, each node should retain a reference to the attributes of the underlying slot. A convenient structured interpretation is:

$$v_i \equiv (sat_i,\, ant_i,\, st_i,\, t_i^{start},\, t_i^{end},\, d_i,\, w_i).$$

This metadata is essential for implementation, debugging, interpretation of the graph, and reconstruction of the selected schedule after optimization.

4.3. Interpretation of the Conflict Graph

The graph is called a conflict graph because edges do not represent physical connectivity or communication links. Instead, they represent incompatibilities.

More precisely:

- selecting a node means selecting the associated candidate slot,
- an edge means that the two corresponding slots cannot both be selected,
- a feasible schedule corresponds to a set of selected nodes containing no edges among them.

Such a set is exactly an *independent set* of the graph. Since the graph also carries node weights, the resulting optimization problem becomes a Maximum Weight Independent Set problem.

4.4. Edge Definition

The most important part of Step 4 is the definition of the conflict edges.

For two distinct candidate slots s_i and s_j, the corresponding vertices v_i and v_j are connected by an edge if and only if the two slots are pairwise incompatible under the current model.

Formally,

$$(v_i, v_j) \in E \quad \Longleftrightarrow \quad s_i \text{ and } s_j \text{ are pairwise incompatible.}$$

In the first version of the model, pairwise incompatibility is defined using the two basic scheduling constraints introduced earlier:

- temporal overlap on the same antenna,
- insufficient reconfiguration or setup time on the same antenna.

4.4.1 Step 4.1: Overlap Conflict on the Same Antenna

Two slots cannot both be selected if they use the same antenna and overlap in time.

Let s_i and s_j be two candidate slots. If

$$ant_i = ant_j$$

and

$$[t_i^{start}, t_i^{end}] \cap [t_j^{start}, t_j^{end}] \neq \emptyset,$$

then the two slots are incompatible and an edge is added:

$$(v_i, v_j) \in E.$$

This is the most immediate type of conflict and captures the fact that one antenna cannot serve two overlapping contacts simultaneously.

4.4.2 Step 4.2: Reconfiguration-Time Conflict on the Same Antenna

Even when two slots do not overlap, they may still be incompatible if the time separating them is shorter than the reconfiguration or setup time required by the antenna.

Let R_{ant} denote the required reconfiguration time associated with antenna ant. For two slots assigned to the same antenna, the forward transition from s_i to s_j is feasible only if

$$t_j^{start} - t_i^{end} \geq R_{ant_i} \qquad \text{whenever } t_i^{end} \leq t_j^{start}.$$

Symmetrically, the transition from s_j to s_i is feasible only if

$$t_i^{start} - t_j^{end} \geq R_{ant_i} \qquad \text{whenever } t_j^{end} \leq t_i^{start}.$$

Therefore, for two slots using the same antenna, an edge is added whenever the temporal separation between them is smaller than the required reconfiguration time.

To write this compactly, define the pairwise time gap:

$$\mathrm{gap}(s_i, s_j) = \begin{cases} t_j^{start} - t_i^{end}, & \text{if } t_i^{end} \le t_j^{start}, \\ t_i^{start} - t_j^{end}, & \text{if } t_j^{end} \le t_i^{start}, \\ 0, & \text{if the intervals overlap.} \end{cases}$$

Then, if

$$ant_i = ant_j \qquad \text{and} \qquad \mathrm{gap}(s_i, s_j) < R_{ant_i},$$

the two slots are incompatible and an edge is added:

$$(v_i, v_j) \in E.$$

This conflict rule captures the practical fact that an antenna cannot instantaneously switch between two different contacts if preparation, steering, or configuration time is required.

4.5. Formal Edge-Construction Rule

Combining the previous definitions, the conflict graph is constructed as follows.

For each pair of distinct slots $s_i, s_j \in S^\star$, add an edge (v_i, v_j) if and only if at least one of the following holds:

$$ant_i = ant_j \quad \text{and} \quad [t_i^{start}, t_i^{end}] \cap [t_j^{start}, t_j^{end}] \neq \emptyset,$$

or

$$ant_i = ant_j \quad \text{and} \quad \mathrm{gap}(s_i, s_j) < R_{ant_i}.$$

Equivalently, the edge set may be written as

$$E = \left\{ (v_i, v_j) \in V \times V : i \neq j,\ \left[ant_i = ant_j \wedge \left([t_i^{start}, t_i^{end}] \cap [t_j^{start}, t_j^{end}] \neq \emptyset \ \vee\ \mathrm{gap}(s_i, s_j) < R_{ant_i} \right) \right] \right\}.$$

This definition is intentionally restricted to pairwise incompatibilities on the same antenna, because these are the conflicts most directly aligned with the current simplified scheduling model and with the future MWIS formulation.

4.6. Undirected Nature of the Graph

The conflict graph is undirected because incompatibility is symmetric. If slot s_i conflicts with slot s_j, then slot s_j also conflicts with slot s_i.

Therefore,

$$(v_i, v_j) \in E \quad \Longleftrightarrow \quad (v_j, v_i) \in E.$$

This is important because the MWIS problem is defined on undirected graphs.

4.7. Resulting MWIS Interpretation

Once the conflict graph has been constructed, the scheduling problem can be interpreted as selecting a subset of nodes such that no two selected nodes are connected by an edge.

This means that a feasible set of selected slots corresponds exactly to an independent set of the graph. Since each node carries a weight, the resulting optimization problem is:

Select an independent set of maximum total weight.

This is the Maximum Weight Independent Set problem induced by the graph $G = (V, E, W)$.

If a binary variable x_i is associated with each node v_i, where

$$x_i = \begin{cases} 1 & \text{if node } v_i \text{ is selected,} \\ 0 & \text{otherwise,} \end{cases}$$

then the induced MWIS formulation is

$$\max \sum_{v_i \in V} W(v_i) x_i$$

subject to

$$x_i + x_j \leq 1 \qquad \forall (v_i, v_j) \in E,$$

and

$$x_i \in \{0, 1\} \qquad \forall v_i \in V.$$

Although this optimization model will be used formally in a later step, it is useful to state it here because Step 4 is the precise point at which the original scheduling problem becomes a graph-based combinatorial optimization problem.

4.8. Graph Diagnostics

Conflict-graph construction should not end with the graph itself. It is also important to compute basic graph statistics, because these quantities are useful for:

- understanding the difficulty of the instance,
- identifying highly congested regions,
- selecting subproblems for later QPU experiments,
- and benchmarking the effect of the reduction pipeline.

At minimum, the following diagnostics should be recorded.

(a) Number of Nodes

$$|V|.$$

(b) Number of Edges

$$|E|.$$

(c) Average Degree

$$\bar{d} = \frac{1}{|V|} \sum_{v \in V} \deg(v).$$

(d) Graph Density

$$\delta = \frac{2|E|}{|V|(|V|-1)}.$$

(e) Connected Components

The connected components of the conflict graph are particularly relevant because disconnected components may later be treated as independent subproblems.

(f) Degree Distribution

The degree distribution helps identify highly conflicted nodes and strongly congested local regions of the instance.

These diagnostics are important not only for analysis, but also for the future identification of graph structures that are more or less suitable for QUBO generation and quantum annealing.

4.9. Implementation View of Step 4

From an implementation perspective, Step 4 can be described as follows.

Input. A reduced candidate-slot set

$$S^\star = \{(s_i, w_i)\}_{i \in I^\star}.$$

Output. A weighted undirected conflict graph

$$G = (V, E, W).$$

Procedure.

1. Create one node v_i for each slot $s_i \in S^\star$.
2. Assign weight $W(v_i) = w_i$.
3. For each pair of slots (s_i, s_j) with $i < j$:
   - add an edge if they use the same antenna and overlap in time,
   - otherwise add an edge if they use the same antenna and the temporal gap is smaller than the required reconfiguration time.
4. Compute the graph diagnostics listed above.

This procedure is sufficiently explicit to be implemented directly once the data structures from the earlier steps are available.

4.10. Modeling Assumption Behind Step 4

The validity of the conflict-graph approach depends on one key assumption:

> The important scheduling conflicts of the simplified model can be represented as fixed pairwise incompatibilities between candidate slots.

This assumption is what allows the scheduling problem to be represented as a static undirected weighted conflict graph. If later phases of the project reveal important higher-order or sequence-dependent constraints that cannot be reduced to pairwise incompatibilities, then either the graph model will need to be refined or those constraints will need to be handled separately in a hybrid postprocessing stage.

For the first model, however, the conflict-graph representation is both appropriate and directly aligned with the intended MWIS/QUBO pipeline.

4.11. Result of Step 4

At the end of this stage, the reduced scheduling instance has been converted into a weighted conflict graph

$$G = (V, E, W),$$

where:

- each node represents one surviving candidate contact slot,
- each node weight represents its operational value,
- each edge represents pairwise incompatibility due to temporal overlap or insufficient reconfiguration time on the same antenna.

In words, the result of Step 4 can be summarized as follows:

> Starting from the reduced candidate-slot set produced by the classical reduction stage, construct an undirected weighted conflict graph in which nodes represent candidate contact slots and edges represent pairwise scheduling conflicts. This graph is the mathematical object that defines the MWIS instance to be used in the following stages.

Final Interpretation of Step 4

Step 4 is the point at which the reduced scheduling problem becomes a graph-based combinatorial optimization problem. From this stage onward, the problem is no longer expressed only in terms of candidate slots and operational data, but as a weighted conflict graph that can be studied through MWIS, translated into QUBO form, and eventually processed by a quantum annealer.

5. Step 5: Validation of the MWIS Abstraction

After Step 4, the reduced scheduling instance has been transformed into a weighted conflict graph

$$G = (V, E, W),$$

where each node represents a surviving candidate contact slot and each edge represents a pairwise incompatibility between two slots.

At this point, the scheduling problem has been reformulated as a Maximum Weight Independent Set (MWIS) problem. However, this reformulation relies on a key modeling assumption:

> The important scheduling restrictions of the simplified problem can be represented as fixed pairwise conflicts between candidate slots.

Step 5 is introduced to validate this assumption before moving to the later stages of the pipeline, such as classical MWIS solving, QUBO construction, and quantum annealing.

The purpose of this section is therefore:

> Evaluate whether the conflict-graph abstraction is a sufficiently faithful representation of the original simplified scheduling problem.

This step is important because an elegant graph formulation is useful only if it preserves the feasibility logic and objective structure of the scheduling model with acceptable accuracy.

5.1. Purpose of the Validation Stage

The validation stage is not intended to replace the graph formulation. Rather, its purpose is to measure how well the graph-based MWIS abstraction captures the scheduling problem defined in the previous sections.

In particular, the following questions must be addressed:

- Does a feasible independent set of the conflict graph correspond to a feasible schedule under the simplified operational model?
- Does the MWIS solution achieve an objective value close to that of a richer reference formulation on the same instance?
- Are there important structural effects that are not captured by the pairwise conflict representation?

If the answer to these questions is positive on representative small instances, then the MWIS abstraction is justified as the basis for the later QUBO and quantum annealing stages.

5.2. Meaning of a Faithful Abstraction

The conflict-graph formulation is considered faithful if, on representative reduced instances, it satisfies the following properties:

1. **Feasibility preservation:** solutions selected through the conflict graph correspond to schedules that are feasible under the simplified operational constraints.
2. **Objective-value consistency:** the total weight of the graph-based solution is close to the value obtained by a richer reference model on the same instance.
3. **Structural adequacy:** no important scheduling effect is systematically omitted by the pairwise conflict representation.

This definition is intentionally practical. The goal is not to prove that the MWIS abstraction is perfect, but to assess whether it is sufficiently accurate to justify the later optimization pipeline.

5.3. Reference Benchmark Model

To validate the MWIS abstraction, a richer reference model is needed. The purpose of this reference model is to provide a benchmark against which the graph-based formulation can be compared on small instances.

The reference model is defined on the same reduced candidate-slot set

$$S^\star = \{(s_i, w_i)\}_{i \in I^\star}$$

used in Step 4. Let

$$x_i = \begin{cases} 1 & \text{if slot } s_i \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}$$

The benchmark objective is:

$$\max \sum_{i \in I^\star} w_i x_i.$$

The reference model enforces the same simplified operational constraints used in the graph formulation, but it is expressed directly as a binary scheduling model rather than through the graph abstraction.

5.3.1 Constraint 1: Overlap Exclusion on the Same Antenna

For every pair of slots $s_i, s_j \in S^\star$ such that

$$ant_i = ant_j$$

and

$$[t_i^{start}, t_i^{end}] \cap [t_j^{start}, t_j^{end}] \neq \emptyset,$$

the following constraint is imposed:

$$x_i + x_j \leq 1.$$

5.3.2 Constraint 2: Reconfiguration-Time Exclusion on the Same Antenna

For every pair of slots $s_i, s_j \in S^\star$ such that

$$ant_i = ant_j$$

and

$$\text{gap}(s_i, s_j) < R_{ant_i},$$

the following constraint is imposed:

$$x_i + x_j \leq 1.$$

Recall that

$$\text{gap}(s_i, s_j) = \begin{cases} t_j^{start} - t_i^{end}, & \text{if } t_i^{end} \leq t_j^{start}, \\ t_i^{start} - t_j^{end}, & \text{if } t_j^{end} \leq t_i^{start}, \\ 0, & \text{if the intervals overlap.} \end{cases}$$

5.3.3 Reference Formulation

The resulting reference model is:

$$\max \sum_{i \in I^\star} w_i x_i$$

subject to

$$x_i + x_j \leq 1 \qquad \forall (i,j) \in \mathcal{C},$$

and

$$x_i \in \{0,1\} \qquad \forall i \in I^\star,$$

where $\mathcal{C}$ is the set of all incompatible slot pairs under the simplified operational model.

This reference model is intentionally defined on the same reduced slot set as the graph. In the current simplified version, the benchmark may appear close to the MWIS model. This is acceptable, because the purpose of the validation is not only to compare objective values, but also:

- to check the correctness of the graph construction,
- to verify that the pairwise conflict logic is properly encoded,
- and to identify whether additional scheduling effects arise when interpreting graph solutions back in operational terms.

5.4. Validation Dimensions

The validation of the MWIS abstraction is organized along three dimensions.

5.4.1 5.1 Feasibility Validation

The first validation question is whether the solution obtained from the graph is feasible when mapped back to the scheduling interpretation.

Let X_{MWIS} denote the set of selected slots induced by the MWIS solution. Then X_{MWIS} should satisfy:

- no temporal overlap on the same antenna,
- no reconfiguration-time violations on the same antenna,
- no violation of the assumptions used in the previous sections.

If a graph-based solution fails this check, then the conflict graph is not correctly representing the scheduling constraints.

5.4.2 5.2 Objective-Value Validation

The second validation question concerns the quality of the graph-based solution.

Let:

$$z_{MWIS} = \text{objective value of the MWIS solution},$$

$$z_{REF} = \text{objective value of the reference benchmark solution}.$$

A natural relative gap measure is:

$$\text{Gap} = \frac{z_{REF} - z_{MWIS}}{z_{REF}},$$

whenever $z_{REF} > 0$.

A small gap indicates that the MWIS abstraction is preserving the objective structure of the scheduling problem well.

5.4.3 5.3 Structural Validation

The third validation question concerns the structure of the model.

Even if the objective values are close, the graph abstraction may still omit important scheduling effects. Therefore, the following issues must be examined:

- whether some constraints are actually higher-order rather than pairwise,
- whether some local sequencing effects are not captured by the static graph,
- whether important resource interactions exist beyond the current same-antenna conflict model.

This part of the validation is partly qualitative, but it is essential because it determines whether the graph model is merely convenient or genuinely adequate.

5.5. Validation Procedure

The MWIS abstraction is validated on representative small instances derived after Step 4.

For each test instance, the following procedure is applied:

1. Construct the reduced candidate-slot set $S^\star$ and the conflict graph $G = (V, E, W)$.
2. Solve the graph instance as an MWIS problem.
3. Solve the same scheduling instance with the reference benchmark formulation.
4. Map the MWIS solution back to the selected slot set.
5. Compare:
    - feasibility of the MWIS solution,
    - objective value of the MWIS solution,
    - objective value of the reference solution,
    - structural consistency between the two solutions.

This procedure makes it possible to determine whether the graph model is both operationally meaningful and optimization-relevant.

5.6. Validation Outputs

For each tested instance, the following quantities should be recorded:

- the MWIS solution value z_{MWIS},
- the reference benchmark value z_{REF},
- the relative gap Gap,
- the feasibility status of the MWIS solution under the scheduling interpretation,
- any detected structural mismatch between the graph abstraction and the scheduling model.

These outputs provide both quantitative and qualitative evidence regarding the adequacy of the conflict-graph formulation.

5.7. Interpretation of the Validation Results

The outcome of Step 5 can be interpreted in two main ways.

Case A: The MWIS Abstraction is Adequate

If the MWIS solution:

- is feasible under the scheduling interpretation,
- achieves a value close to the reference benchmark,
- and reveals no major structural mismatch,

then the conflict graph is considered an adequate abstraction of the simplified scheduling problem.

In this case, the project can proceed consistently to:

- classical MWIS solution methods,
- QUBO formulation,
- and quantum annealing experiments.

Case B: The MWIS Abstraction is Incomplete

If the MWIS solution:

- fails feasibility checks,
- exhibits a large value gap,
- or reveals structural effects not captured by pairwise conflicts,

then the graph abstraction is incomplete.

In that case, the conclusion is not necessarily that the approach must be abandoned. Rather, it indicates that:

- the MWIS graph may still capture the conflict-dense core of the problem,
- but some additional constraints may need to be handled separately,
- for example through a later repair stage, postprocessing, or a more hybrid classical–quantum workflow.

5.8. Role of Step 5 in the Overall Pipeline

This step plays a strategic role in the project because it provides a bridge between modeling and optimization.

Without validation, the graph formulation would remain only a plausible abstraction. With validation, it becomes a justified representation whose strengths and limitations are explicitly understood.

Therefore, Step 5 is the stage that turns the conflict-graph model from a convenient reformulation into a tested abstraction suitable for the later MWIS/QUBO pipeline.

5.9. Result of Step 5

At the end of this stage, the project obtains an explicit assessment of whether the conflict-graph formulation is an adequate approximation of the simplified scheduling problem.

In words, the result of Step 5 can be summarized as follows:

> Compare the MWIS solution induced by the conflict graph against a richer reference benchmark on representative small instances, in order to evaluate feasibility preservation, objective-value consistency, and structural adequacy of the graph abstraction.

Final Interpretation of Step 5

Step 5 validates whether the conflict graph built in the previous stage is a sufficiently faithful mathematical representation of the simplified scheduling problem. This validation is essential before using the graph as the basis for MWIS solving, QUBO construction, and quantum annealing experiments.

6. Step 6: Classical Solution of the MWIS Instance

After Step 5, the conflict-graph abstraction has been validated as an acceptable representation of the simplified scheduling problem. The next step is therefore to solve the resulting graph instance classically before proceeding to QUBO construction and quantum annealing.

The purpose of Step 6 is:

> Solve the weighted conflict graph as a Maximum Weight Independent Set problem using classical optimization methods, and use the resulting solutions as the main benchmark for the later quantum stages.

This stage is important for three reasons:

- it provides an exact or near-exact classical baseline,
- it verifies that the graph pipeline can be solved end-to-end,
- and it identifies which instances are easy or difficult before any QUBO or quantum annealing experiments are attempted.

6.1. Purpose of the Classical MWIS Stage

Once the reduced scheduling problem has been transformed into a weighted conflict graph

$$G = (V, E, W),$$

the natural next step is to solve the induced MWIS problem directly.

This classical solution stage does not replace the future quantum formulation. Rather, it serves as the benchmark against which the later QUBO and quantum annealing results will be compared.

In particular, Step 6 allows the project to:

- obtain a reference solution value for each graph instance,
- reconstruct the corresponding selected schedule,

- measure runtime and instance difficulty,
- and establish whether the graph instances are genuinely challenging enough to motivate the later quantum stage.

6.2. MWIS Formulation to be Solved

Let one binary variable x_i be associated with each node $v_i \in V$, where

$$x_i = \begin{cases} 1 & \text{if node } v_i \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}$$

The MWIS problem induced by the graph $G = (V, E, W)$ is:

$$\max \sum_{v_i \in V} W(v_i) x_i$$

subject to

$$x_i + x_j \leq 1 \qquad \forall (v_i, v_j) \in E,$$

and

$$x_i \in \{0, 1\} \qquad \forall v_i \in V.$$

This formulation ensures that no two adjacent vertices can be selected simultaneously, and therefore any feasible solution corresponds to an independent set of the graph. The objective maximizes the total weight of the selected nodes.

6.3. Classical Solution Method

In the proposed pipeline, the MWIS instance is solved classically through an exact binary linear optimization model.

This means that, although the original project is not based on a full MILP formulation of the entire scheduling problem, the reduced MWIS graph is solved using a classical binary optimization solver. This choice is appropriate because:

- the MWIS formulation is already linear in the binary variables,
- it provides an exact benchmark on small and medium instances,
- it is straightforward to implement,
- and it produces a strong reference for later comparison with the QUBO and quantum annealing stages.

Therefore, Step 6 uses a classical integer-programming solver to solve the graph-induced MWIS problem exactly whenever possible.

Important Clarification

At this stage, the binary linear solver is not being used to solve the original scheduling problem directly. Instead, it is being used to solve the MWIS problem induced by the reduced conflict graph. This distinction is important because the project remains graph-first and quantum-oriented, while still relying on a classical exact solver for benchmarking.

6.4. Optimal Classical MWIS Solution

Let

$$x^\star = (x_1^\star, x_2^\star, \ldots, x_{|V|}^\star)$$

denote an optimal classical solution of the MWIS formulation.

The corresponding optimal classical objective value is:

$$z_{MWIS}^{class} = \sum_{v_i \in V} W(v_i) x_i^\star.$$

The selected node set is:

$$V^\star = \{\, v_i \in V : x_i^\star = 1 \,\}.$$

This set $V^\star$ is an independent set of the conflict graph and therefore corresponds to a feasible selection of non-conflicting candidate contact slots under the simplified model.

6.5. Reconstruction of the Scheduling Solution

The graph solution must then be interpreted back in operational terms.

Since each node v_i corresponds to one candidate slot s_i, the selected slot set induced by the classical MWIS solution is:

$$S_{sel}^\star = \{\, s_i : x_i^\star = 1 \,\}.$$

In words, the final classical schedule associated with the MWIS solution is obtained simply by selecting all candidate slots whose corresponding nodes are active in the optimal graph solution.

This reconstruction step is important because it restores the meaning of the graph-theoretic solution in terms of:

- satellite contacts,
- antenna assignments,
- start and end times,
- and operational value.

Therefore, Step 6 should always report both:

- the graph solution,
- and the reconstructed selected slot set.

6.6. Feasibility of the Reconstructed Schedule

Because the graph has been constructed from pairwise incompatibilities, any independent set of the graph should correspond to a feasible schedule under the simplified model.

However, the reconstructed schedule should still be checked explicitly against the modeled constraints:

- no overlap on the same antenna,
- no reconfiguration-time violations on the same antenna,
- consistency with the assumptions of Sections 1 to 4.

This check serves two purposes:

- it confirms that the graph construction is correct,
- and it provides a final operational interpretation of the classical MWIS solution.

6.7. Quantities to be Recorded

The classical MWIS stage should produce more than just one selected solution. For each instance, the following quantities should be recorded.

(a) Classical MWIS Objective Value

$$z_{MWIS}^{class}.$$

(b) Runtime

The wall-clock time required by the classical solver to obtain the MWIS solution.

(c) Number of Selected Nodes

$$|V^\star|.$$

(d) Number of Selected Contact Slots

Since each selected node corresponds to one selected slot, this is

$$|S_{sel}^\star|.$$

(e) Feasibility Status of the Reconstructed Schedule

Whether the selected slot set satisfies all modeled scheduling constraints.

(f) Graph Descriptors of the Instance

The following graph descriptors should be stored together with the classical solution result:

- number of nodes $|V|$,
- number of edges $|E|$,
- average degree,
- graph density,
- number of connected components.

These descriptors are important because they help explain the runtime and difficulty of the instance.

6.8. Role of Step 6 in the Full Pipeline

Step 6 plays a central role in the overall project because it establishes the main classical baseline for all later stages.

In particular, the classical MWIS results obtained here will later be used to compare:

- objective values produced by QUBO-based methods,
- runtime and scalability,
- solution quality of the quantum annealing stage,
- and the quality of any hybrid classical–quantum refinement strategy.

Thus, Step 6 is not simply an auxiliary computation. It is the reference optimization stage for the graph model itself.

6.9. Suggested Computational Procedure

From an implementation perspective, the classical MWIS stage may be summarized as follows.

Input. A weighted conflict graph

$$G = (V, E, W).$$

Output.

- the optimal classical binary solution $x^\star$,
- the objective value z_{MWIS}^{class},
- the selected node set $V^\star$,
- the reconstructed selected slot set $S_{sel}^\star$,
- and the associated runtime and graph statistics.

Procedure.

1. Formulate the MWIS instance as a binary linear optimization problem.
2. Solve it using a classical exact integer-programming solver.
3. Extract the selected node set $V^\star$.
4. Map the selected nodes back to the original candidate slots.
5. Verify feasibility of the reconstructed schedule.
6. Record objective value, runtime, and graph descriptors.

This procedure is sufficiently explicit to be implemented directly once the graph-construction stage is complete.

6.10. Interpretation of the Classical Solution

The result of this stage should be interpreted in two complementary ways.

Graph-Theoretic Interpretation

The solution is an independent set of maximum weight in the conflict graph.

Scheduling Interpretation

The solution is a set of non-conflicting candidate contact slots selected from the reduced scheduling instance.

These two interpretations are equivalent under the graph abstraction established in the previous steps. Their simultaneous use is important because:

- the graph interpretation is necessary for MWIS and QUBO reasoning,
- while the scheduling interpretation is necessary for operational meaning and later evaluation.

6.11. Result of Step 6

At the end of this stage, each reduced conflict graph instance has been solved classically as an MWIS problem, and the resulting optimal solution has been mapped back into a selected set of candidate contact slots.

In words, the outcome of Step 6 can be summarized as follows:

> Solve the weighted conflict graph with a classical exact MWIS solver, reconstruct the corresponding set of selected contact slots, and record the objective value, feasibility, runtime, and graph characteristics as the main benchmark for the later QUBO and quantum annealing stages.

Final Interpretation of Step 6

Step 6 is the main classical optimization stage of the graph-based pipeline. It provides the benchmark against which the later QUBO and quantum annealing results will be evaluated, while also confirming that the reduced conflict graph can be solved and interpreted as a valid scheduling solution.

7. Step 7: QUBO Formulation of the MWIS Instance

After Step 6, the reduced conflict graph has already been solved classically, and the resulting solutions provide the main benchmark for the later quantum stages. The next step is therefore to translate the graph-based MWIS problem into a Quadratic Unconstrained Binary Optimization (QUBO) formulation, which is the optimization form required by quantum annealing hardware.

The purpose of Step 7 is:

> Transform the weighted conflict graph into a QUBO model whose low-energy configurations correspond to high-value feasible independent sets of the graph.

This step is the direct mathematical bridge between the graph formulation and the quantum annealing stage. In particular:

- Step 4 produced the weighted conflict graph,
- Step 6 solved the corresponding MWIS instance classically,
- Step 7 now converts that same MWIS instance into a QUBO representation.

7.1. Purpose of the QUBO Formulation

Quantum annealers are designed to minimize an energy function over binary variables. Therefore, in order to use quantum annealing, the MWIS problem must be rewritten as an unconstrained quadratic binary objective.

The QUBO formulation must preserve two essential aspects of the MWIS problem:

1. selecting high-value nodes should reduce the energy,
2. selecting two conflicting nodes simultaneously should increase the energy.

The first requirement encodes the optimization objective, while the second requirement encodes feasibility through penalty terms.

Thus, the role of Step 7 is not to define a new optimization problem, but to reformulate the already-defined MWIS problem in a way that is compatible with quantum annealing.

7.2. Binary Decision Variables

Let one binary variable x_i be associated with each graph node $v_i \in V$, where

$$x_i = \begin{cases} 1 & \text{if node } v_i \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}$$

Since each node corresponds to one candidate contact slot, $x_i = 1$ means that the corresponding slot is selected in the final schedule.

These are exactly the same binary variables used in the classical MWIS formulation. What changes in Step 7 is not the meaning of the variables, but the mathematical form of the optimization problem.

7.3. MWIS Objective and Conflict Structure

Recall that the MWIS problem on the weighted conflict graph $G = (V, E, W)$ is:

$$\max \sum_{v_i \in V} W(v_i) x_i$$

subject to

$$x_i + x_j \leq 1 \qquad \forall (v_i, v_j) \in E,$$

and

$$x_i \in \{0, 1\} \qquad \forall v_i \in V.$$

The QUBO formulation must encode both the node weights and the conflict constraints inside a single unconstrained quadratic objective function.

7.4. QUBO Formulation of MWIS

A standard QUBO representation of MWIS is:

$$\min \left(- \sum_{v_i \in V} W(v_i) x_i + M \sum_{(v_i, v_j) \in E} x_i x_j \right),$$

where:

- the first term rewards the selection of high-value nodes,
- the second term penalizes the simultaneous selection of conflicting nodes,
- $M > 0$ is a penalty coefficient.

This formulation has the intended interpretation:

- selecting a node decreases the objective by its weight,
- selecting two adjacent nodes increases the objective by a penalty term.

Therefore, a low-energy solution is encouraged to include valuable nodes while avoiding conflicts.

7.5. Interpretation of the Two Terms

The two parts of the QUBO play complementary roles.

7.5.1 Reward Term

The linear term

$$-\sum_{v_i \in V} W(v_i) x_i$$

encourages the annealer to select nodes with large weight. Since the problem is written as a minimization, a large positive node weight produces a large negative contribution when the corresponding variable is set to 1.

Thus, higher-value contact slots become more attractive in the energy landscape.

7.5.2 Penalty Term

The quadratic term

$$M \sum_{(v_i, v_j) \in E} x_i x_j$$

discourages infeasible solutions by penalizing every pair of conflicting nodes that are selected together.

If both $x_i = 1$ and $x_j = 1$ for an edge $(v_i, v_j) \in E$, then the term $x_i x_j$ equals 1 and contributes a penalty of magnitude M.

Thus, this term enforces the independent-set structure through energetic discouragement rather than through explicit constraints.

7.6. Penalty Coefficient Selection

The penalty coefficient M is one of the most important parameters of the QUBO model.

It must be large enough so that violating a conflict edge is energetically unattractive, but not so large that the coefficients become badly scaled for the annealing process.

In general:

- if M is too small, infeasible solutions may be favored because the gain from selecting conflicting nodes may outweigh the penalty,
- if M is too large, the objective may become dominated by penalties and the numerical scaling may become less suitable for the annealer.

A common sufficient choice is to select M larger than the largest node weight in the instance:

$$M > \max_{v_i \in V} W(v_i).$$

In practice, however, the exact value of M may need to be tuned experimentally, especially when coefficient normalization or hardware limitations are taken into account.

7.7. QUBO Matrix Representation

The QUBO objective can also be written in matrix form:

$$\min x^T Q x,$$

where

$$x = (x_1, x_2, \ldots, x_{|V|})^T$$

is the binary decision vector and Q is a symmetric matrix encoding both rewards and penalties.

For the MWIS formulation:

- the diagonal entries encode the linear reward terms,
- the off-diagonal entries encode the conflict penalties.

More explicitly, one may define:

$$Q_{ii} = -W(v_i) \qquad \forall v_i \in V,$$

and

$$Q_{ij} = \begin{cases} M, & \text{if } (v_i, v_j) \in E,\ i < j, \\ 0, & \text{otherwise.} \end{cases}$$

Up to standard conventions for symmetric QUBO storage, this matrix fully defines the optimization problem to be sent to the quantum annealer.

7.8. Relationship Between the QUBO and the Graph

The QUBO is a direct algebraic encoding of the weighted conflict graph:

- each graph node becomes one binary variable,
- each node weight becomes a linear coefficient,
- each graph edge becomes a quadratic coupling penalty.

This correspondence is important because it shows that the QUBO is not an additional model layered on top of the graph. It is simply a different mathematical representation of the same reduced scheduling instance.

Thus, the structure of the QUBO is inherited directly from the structure of the conflict graph.

7.9. Feasible and Infeasible Configurations in the QUBO

A binary configuration x is:

- **feasible** if the set of selected nodes forms an independent set of the graph,

- **infeasible** if at least one conflict edge has both endpoints selected.

In the QUBO formulation, infeasibility is not forbidden explicitly. Instead, it is penalized in the energy function. Therefore:

- feasible solutions are expected to appear at low energy,
- infeasible solutions may still appear, especially if M is poorly chosen or the annealer does not find the true global minimum.

For this reason, later stages of the pipeline should explicitly check the feasibility of the returned solutions.

7.10. Consistency with the Classical MWIS Stage

The QUBO formulation of Step 7 is built from the same weighted conflict graph used in Step 6. Therefore, both stages solve the same reduced combinatorial problem, but in different representations:

- Step 6 solves it as a classical exact MWIS problem,
- Step 7 reformulates it as an unconstrained quadratic binary minimization problem.

This consistency is essential because it allows the later quantum results to be compared directly against the classical MWIS benchmark.

7.11. Computational Quantities to Record

When building the QUBO instance, it is useful to record several quantities that describe its size and structure.

At minimum, the following should be stored:

- the number of binary variables,
- the number of nonzero quadratic couplings,
- the penalty coefficient M,
- the range of diagonal coefficients,
- the range of off-diagonal coefficients.

These quantities are relevant because they help evaluate:

- the compactness of the formulation,
- the effect of reduction and graph sparsity,
- and the expected difficulty of embedding the instance on the target annealing hardware.

7.12. Implementation View of Step 7

From an implementation perspective, the QUBO construction stage may be described as follows.

Input. A weighted conflict graph

$$G = (V, E, W).$$

Output. A QUBO model, represented for example by:

- a binary variable vector x,
- a QUBO matrix Q,
- and a chosen penalty coefficient M.

Procedure.

1. Associate one binary variable x_i with each node $v_i \in V$.
2. Set the diagonal coefficient of each variable according to the node weight:

$$Q_{ii} = -W(v_i).$$

3. For each graph edge $(v_i, v_j) \in E$, set the corresponding quadratic penalty coefficient:

$$Q_{ij} = M.$$

4. Record the size and coefficient statistics of the resulting QUBO instance.

This procedure is sufficiently explicit to be implemented directly once the conflict graph has been constructed.

7.13. Interpretation of Step 7 in the Full Pipeline

Step 7 is the point at which the reduced scheduling problem becomes compatible with quantum annealing.

The previous stages have already:

- defined the scheduling problem,
- generated candidate contact slots,
- reduced the candidate set,
- built the conflict graph,
- validated the MWIS abstraction,
- and solved the graph classically.

Now, Step 7 prepares the same reduced graph instance for the quantum hardware by rewriting it in QUBO form.

Thus, this stage is the final mathematical translation required before the project can move from graph optimization to annealing-based execution.

7.14. Result of Step 7

At the end of this stage, the weighted conflict graph has been translated into a QUBO instance of the form

$$\min x^T Q x,$$

whose low-energy binary configurations correspond to high-value feasible independent sets of the graph.

In words, the result of Step 7 can be summarized as follows:

Translate the weighted conflict graph into a QUBO model by assigning one binary variable to each graph node, one linear reward term to each node weight, and one quadratic penalty term to each conflict edge, so that the resulting energy function can be processed by a quantum annealer.

Final Interpretation of Step 7

Step 7 is the mathematical translation stage that turns the reduced MWIS graph instance into an annealer-compatible optimization problem. From this point onward, the project no longer works only with graph-theoretic objects, but with a QUBO energy function that can be embedded and executed on quantum annealing hardware.

8. Step 8: Selection of QPU-Suitable Subproblems

After Step 7, the reduced scheduling instance has been translated into a QUBO model. However, even after the reduction and graph-construction stages, not every QUBO instance will be directly suitable for execution on a quantum annealer.

This limitation arises because current quantum annealing hardware has finite resources and restricted connectivity. Therefore, before attempting execution on the QPU, it is necessary to identify which parts of the reduced problem are sufficiently compact and structurally appropriate for embedding and annealing.

The purpose of Step 8 is:

> Select the subproblems of the reduced conflict graph and corresponding QUBO that are most suitable for execution on the target quantum annealing hardware.

This step is necessary because the goal of the project is not simply to generate a QUBO, but to generate QUBO instances that can be meaningfully executed and benchmarked on actual annealing hardware.

8.1. Purpose of the Subproblem-Selection Stage

The reduced conflict graph obtained in the previous stages may still be too large, too dense, or too irregular for direct QPU execution. Therefore, the full reduced instance must be analyzed and, if necessary, decomposed into smaller candidate subproblems.

The role of Step 8 is to identify subproblems that satisfy two conditions:

1. they remain meaningful from the scheduling perspective,
2. they are sufficiently small and structured for QPU execution.

Thus, Step 8 is not an optimization stage by itself. Instead, it is a hardware-oriented instance-selection and decomposition stage.

8.2. Why Subproblem Selection is Necessary

The QUBO produced in Step 7 is a valid mathematical formulation of the MWIS problem, but quantum annealing hardware introduces additional practical limitations:

- a limited number of qubits,
- limited physical connectivity between qubits,

- embedding overhead when mapping logical variables to physical qubits,
- and performance degradation on very dense or poorly structured instances.

As a consequence, even a reduced QUBO may still be too large for direct execution. For this reason, the project must explicitly define which subproblems are sent to the QPU and which are handled classically.

This is fully consistent with the hybrid logic of the overall pipeline: the QPU is intended to process the most relevant reduced cores of the instance, rather than the entire global problem without selection.

8.3. Input to Step 8

The input to this stage is the reduced weighted conflict graph and its associated QUBO:

$$G = (V, E, W), \qquad Q.$$

In addition, the graph diagnostics already computed in Step 4 are available:

- number of nodes,
- number of edges,
- average degree,
- graph density,
- connected components,
- degree distribution.

These quantities are essential for deciding which parts of the instance are the most promising for annealing-based execution.

8.4. Candidate Subproblem Definition

A candidate subproblem is defined as a subgraph

$$G_k = (V_k, E_k, W_k)$$

such that:

- $V_k \subseteq V$,
- E_k contains the edges of E induced by V_k,
- W_k is the restriction of the weight function W to the node set V_k.

The corresponding QUBO subproblem is the restriction of the full QUBO to the variables associated with the nodes in V_k.

Thus, each candidate subproblem is itself a smaller weighted conflict graph and a smaller QUBO instance that can be evaluated for quantum suitability.

8.5. Subproblem Selection Criteria

A candidate subproblem is considered suitable for QPU execution if it satisfies a collection of structural and practical criteria. The following are the most relevant ones for the first version

of the project.

8.5.1 8.1 Size Criterion

The first criterion is the size of the subproblem. Let

$$|V_k|$$

denote the number of nodes in candidate subproblem G_k.

Since each graph node corresponds to one QUBO variable, subproblems with smaller $|V_k|$ are generally more suitable for the QPU. This is the most basic requirement.

8.5.2 8.2 Density Criterion

The density of the subproblem is

$$\delta_k = \frac{2|E_k|}{|V_k|(|V_k| - 1)}.$$

Subproblem density is important because dense conflict graphs usually produce denser QUBOs, which are harder to embed directly on sparse annealing hardware topologies.

At the same time, the most conflict-dense regions are often the most interesting from the optimization perspective. Therefore, density must be treated as a tradeoff:

- very low density may indicate an easy problem,
- very high density may indicate an interesting but hard-to-embed problem.

8.5.3 8.3 Connectedness Criterion

Connected components of the conflict graph are natural candidates for decomposition.

If

$$G = G_1 \cup G_2 \cup \cdots \cup G_m$$

is the decomposition of the graph into connected components, then each component may be treated as an independent subproblem, because there are no conflict edges between distinct components.

This makes connected components one of the most natural first decomposition structures in the project.

8.5.4 8.4 Conflict Intensity Criterion

A subproblem should ideally correspond to a region in which many strong alternatives compete for the same resources. Such regions are more likely to be difficult and therefore more relevant for quantum annealing experiments.

Conflict intensity may be estimated through:

- average degree,
- maximum degree,
- local density,
- and concentration of high-value nodes in the same subgraph.

The idea is that the QPU should be reserved for the parts of the instance where the combinatorial competition is strongest.

8.5.5 8.5 Weight-Structure Criterion

The distribution of node weights inside the subproblem also matters. A good candidate subproblem should not consist only of uniformly weak nodes. Instead, it should contain:

- meaningful scheduling alternatives,
- and nontrivial tradeoffs between valuable conflicting choices.

This criterion ensures that the selected subproblems are not only structurally feasible for the hardware, but also relevant from the application point of view.

8.6. Recommended First Decomposition Strategy

For the first implementation, the recommended strategy is to decompose the reduced conflict graph into connected components and use them as the initial candidate subproblems.

This choice is attractive because:

- it is mathematically clean,
- it is easy to implement,
- it preserves exact independence between subproblems,
- and it avoids introducing arbitrary decomposition rules at the first stage.

Therefore, if the connected components of the graph are

$$C_1, C_2, \ldots, C_m,$$

then the first family of candidate subproblems is:

$$G_k = G[C_k], \qquad k = 1, \ldots, m,$$

where $G[C_k]$ denotes the subgraph induced by component C_k.

8.7. Secondary Local Extraction Strategy

If a connected component is still too large, then a secondary extraction step may be applied inside that component.

A practical option is to identify local dense regions around high-conflict nodes. For example:

1. select a node with large degree or high weight,
2. extract its local neighborhood,
3. build the induced subgraph on that neighborhood,
4. and evaluate that induced subgraph as a QPU candidate.

This local extraction strategy is useful when the graph contains one or more large connected components whose full size exceeds the hardware limits.

Thus, the first implementation may use the following hierarchical logic:

- first decompose by connected components,

- then, if necessary, extract smaller local dense subgraphs from oversized components.

8.8. Practical QPU-Suitability Score

To make the selection stage more explicit, each candidate subproblem G_k may be assigned a suitability score. A simple conceptual score can be defined as:

$$\sigma_k = f\big(|V_k|, \delta_k, \bar{d}_k, w_k^{max}, \mathrm{comp}_k\big),$$

where:

- $|V_k|$ is the number of nodes,
- δ_k is the density,
- $\bar{d}_k$ is the average degree,
- w_k^{max} is the maximum node weight,
- comp_k is a structural descriptor such as component size or embedding complexity estimate.

In the first version of the project, this score does not need to be overly sophisticated. Its role is simply to rank candidate subproblems according to:

- likely hardware feasibility,
- and likely optimization relevance.

8.9. Subproblem Selection Procedure

A practical subproblem-selection procedure may be defined as follows.

Input. The reduced conflict graph

$$G = (V, E, W)$$

and its associated QUBO model.

Output. A family of selected subproblems

$$\mathcal{G}_{QPU} = \{G_1, G_2, \ldots, G_r\}$$

that are considered suitable for QPU execution.

Procedure.

1. Compute the connected components of the graph.
2. Treat each connected component as an initial candidate subproblem.
3. For each candidate subproblem, compute:
    - number of nodes,
    - number of edges,
    - density,

- average degree,
- weight statistics.

4. Discard components that are too small to be interesting or too large to be plausible QPU candidates.
5. If a remaining component is still too large, extract smaller local dense subgraphs from it.
6. Rank the resulting candidates according to a QPU-suitability criterion.
7. Retain the highest-priority subproblems for QUBO embedding and annealing experiments.

This procedure is sufficiently explicit to be implemented once the graph-construction stage is available.

8.10. Role of Step 8 in the Hybrid Pipeline

Step 8 is central to the hybrid character of the project.

The full reduced scheduling instance is not assumed to be entirely solvable on the QPU. Instead:

- classical preprocessing reduces the instance,
- classical graph analysis decomposes the instance,
- and only selected compact conflict-dense subproblems are sent to the quantum annealer.

Thus, Step 8 is the stage that determines how the global reduced problem is split between:

- classical handling,
- and quantum execution.

This makes the step essential not only for feasibility on real hardware, but also for the scientific meaning of the hybrid classical–quantum workflow.

8.11. Quantities to Record

For each selected candidate subproblem, the following quantities should be recorded:

- number of nodes $|V_k|$,
- number of edges $|E_k|$,
- density δ_k,
- average degree $\bar{d}_k$,
- connected-component identity or extraction origin,
- maximum and average node weights,
- estimated QUBO size and coupling count.

These data are important because they allow later quantum results to be interpreted in relation to the size and structure of each selected subproblem.

8.12. Interpretation of the Selected Subproblems

Each selected subproblem should be interpreted in two complementary ways.

Graph-Theoretic Interpretation

It is a compact weighted conflict graph extracted from the larger reduced instance.

Scheduling Interpretation

It is a localized region of the scheduling problem containing a set of strongly competing contact opportunities whose resolution is considered important and sufficiently difficult to justify quantum treatment.

This dual interpretation is important because the subproblem must be both:

- structurally meaningful for optimization,
- and operationally meaningful for the original scheduling application.

8.13. Result of Step 8

At the end of this stage, the project has identified a family of reduced graph/QUBO subproblems that are considered suitable for quantum annealing execution.

In words, the outcome of Step 8 can be summarized as follows:

> Analyze the reduced weighted conflict graph, decompose it into candidate subproblems, and retain the compact and conflict-dense subgraphs that are both operationally meaningful and structurally suitable for QPU execution.

Final Interpretation of Step 8

Step 8 is the instance-selection stage that connects the reduced QUBO formulation to the practical limitations of the quantum annealer. Its role is to determine which parts of the reduced scheduling problem are small and meaningful enough to be executed on the QPU within a hybrid classical–quantum pipeline.

9. Step 9: Pure Quantum Annealing Execution

After Step 8, the project has identified a set of reduced QUBO subproblems that are considered suitable for execution on the target quantum annealing hardware. The next step is therefore to run these subproblems on the QPU and evaluate their behavior under pure quantum annealing conditions.

The purpose of Step 9 is:

> Execute the selected reduced QUBO instances on the quantum annealer, study the effect of embedding and annealing parameters, and compare the resulting solutions against the classical MWIS benchmark.

This stage is important because it is the first point in the pipeline where the reduced scheduling problem is actually processed on quantum annealing hardware. In addition, the proposal explicitly motivates an initial pure-annealing phase in order to isolate and study the intrinsic behavior of the QPU on small problem instances before moving to a broader hybrid strategy. *Cf. quARKS - Ground Station*, Section “Proposed quantum approach and classical benchmark.” :contentReference[oaicite:0]index=0

9.1. Purpose of the Pure Annealing Stage

The pure quantum annealing stage serves three main purposes:

1. **Hardware-oriented validation.** It verifies that the selected QUBO instances can be embedded and executed on the target QPU.
2. **Parameter study.** It allows the project to study the effect of chain strength, annealing schedules, and repeated sampling on solution quality.
3. **Benchmarking.** It provides the first direct comparison between QPU-generated solutions and the classical MWIS solutions obtained in Step 6.

This stage is intentionally restricted to reduced subproblems, because the goal is not yet to solve the whole scheduling problem end-to-end on the QPU, but to understand the behavior of the annealer on carefully selected graph cores.

9.2. Input to Step 9

The input to this stage is a family of selected QUBO subproblems:

$$\mathcal{Q}_{QPU} = \{Q_1, Q_2, \ldots, Q_r\},$$

each derived from a selected conflict-graph subproblem

$$\mathcal{G}_{QPU} = \{G_1, G_2, \ldots, G_r\}.$$

For each subproblem Q_k, the following are assumed to be known:

- the set of binary decision variables,
- the QUBO coefficients,
- the graph structure of the associated MWIS instance,
- the classical benchmark solution from Step 6,
- and the graph/QUBO size descriptors computed in earlier stages.

Thus, each QPU execution can later be compared directly against a corresponding classical reference solution.

9.3. Logical and Physical Problem Representation

The QUBO defined in Step 7 is a *logical* problem. Quantum annealing hardware, however, works with a *physical* graph of qubits and couplers. Therefore, before execution, the logical QUBO must be embedded into the physical topology of the QPU.

This distinction is central to Step 9:

- the **logical problem** is the reduced QUBO formulation,
- the **physical problem** is the embedded instance actually run on the annealer.

Consequently, pure annealing execution must explicitly include the embedding stage.

9.4. Step 9.1: Embedding of the Logical QUBO

For each selected QUBO instance Q_k, the first execution step is to compute an embedding onto the target hardware topology.

Let L_k denote the logical interaction graph of the QUBO, and let P denote the physical connectivity graph of the annealer. An embedding is a mapping from logical variables to connected chains of physical qubits such that:

- each logical variable is represented by one chain of qubits,
- couplings between logical variables are realized through physical couplers between the corresponding chains.

The embedding stage produces several quantities that must be recorded:

- whether embedding succeeds or fails,
- the number of physical qubits used,
- the chain lengths,
- the maximum chain length,
- and any embedding-specific diagnostics.

These quantities are important because the difficulty of the annealing problem depends not only on the logical QUBO size, but also on the physical cost of embedding it.

9.5. Step 9.2: Chain Strength Selection

Once a logical variable is represented by a chain of physical qubits, the qubits in the chain must be coupled strongly enough so that they tend to behave consistently during annealing. This is controlled through the chain strength parameter.

If the chain strength is too small:

- chains may break,
- and the physical solution may not correspond reliably to a valid logical binary vector.

If the chain strength is too large:

- the physical energy landscape may become poorly scaled,
- and the annealer may spend too much of its effective resolution enforcing the chains rather than exploring the logical objective.

For this reason, Step 9 should include a systematic chain-strength study. Let

$$\Lambda = \{\lambda_1, \lambda_2, \ldots, \lambda_m\}$$

denote a set of tested chain-strength values. Then each selected subproblem is executed multiple times under different chain-strength settings.

The chain-strength study is a key part of the pure annealing stage because it determines the quality and interpretability of the logical solutions reconstructed from the physical output.

9.6. Step 9.3: Annealing Schedule Selection

After embedding and chain-strength selection, the next parameter family concerns the annealing schedule itself.

The simplest execution mode uses a standard forward anneal. However, the pure annealing stage should also allow controlled experiments with alternative annealing schedules when supported by the target hardware.

The following schedule families are relevant:

- **Standard annealing:** the default forward anneal from the initial state to the final problem Hamiltonian.
- **Annealing pauses:** schedules that include a pause in the evolution, with the goal of improving exploration or thermalization.
- **Reverse annealing:** schedules that start from a classical candidate solution, move partially backward in the annealing path, and then re-anneal locally.

At the first implementation stage, standard annealing should be the default mode. If time and hardware access permit, pauses and reverse annealing can then be studied as controlled extensions.

This design is consistent with the proposal, which explicitly suggests using pure annealing to study embedding, chain strength, and advanced annealing controls before moving to a broader hybrid architecture. :contentReference[oaicite:1]index=1

9.7. Step 9.4: Repeated Sampling

Quantum annealing is not expected to return the optimal solution deterministically in a single run. Therefore, each embedded QUBO instance must be sampled multiple times.

For each selected subproblem, let

$$N_{reads}$$

denote the number of reads or samples collected from the QPU. The role of repeated sampling is:

- to estimate the quality distribution of the returned solutions,
- to identify the best solution found,
- to measure the fraction of feasible solutions,
- and to assess the stability of the annealing process under repeated runs.

Thus, a single QPU experiment for one subproblem is not just one anneal, but a collection of repeated samples obtained under fixed execution settings.

9.8. Logical Solution Reconstruction

Each returned physical sample must be mapped back to a logical binary vector. This reconstruction step is necessary because:

- the QPU returns values at the physical-qubit level,
- while the scheduling interpretation is defined at the logical-variable level.

Let $\tilde{x}$ denote a physical sample returned by the annealer. The reconstruction procedure maps $\tilde{x}$ to a logical binary vector

$$x = (x_1, x_2, \ldots, x_n).$$

If some chains are broken, a chain-resolution rule must be applied, for example:

- majority vote,
- energy-minimizing repair,
- or another deterministic decoding rule supported by the implementation environment.

This reconstructed logical vector is then interpreted as the candidate MWIS solution returned by the annealer.

9.9. Feasibility Check of the Returned Solutions

Because the QUBO encodes conflicts through penalties rather than explicit hard constraints, not every returned logical vector is guaranteed to be feasible.

Therefore, each reconstructed solution must be checked explicitly against the conflict graph.

Let x be a reconstructed logical solution. Then x is feasible if and only if

$$x_i + x_j \leq 1 \qquad \forall (v_i, v_j) \in E_k,$$

where E_k is the edge set of the corresponding subproblem graph.

This feasibility check is essential because:

- it distinguishes valid MWIS solutions from penalized but infeasible samples,
- and it makes it possible to compare the QPU output fairly against the classical MWIS benchmark.

9.10. Objective-Value Evaluation

For each reconstructed logical solution x, the corresponding MWIS objective value is:

$$z(x) = \sum_{v_i \in V_k} W(v_i) x_i.$$

The most important QPU-side objective values to record are:

- the best feasible value found over all reads,
- the average feasible value,
- and the overall best value regardless of feasibility.

Let

$$z_{QPU}^{best}$$

denote the best feasible objective value found for a given subproblem. This quantity will later be compared directly with the classical MWIS benchmark

$$z_{MWIS}^{class}.$$

9.11. Benchmarking Against the Classical Solution

The most important comparison of Step 9 is between:

- the classical MWIS solution obtained in Step 6,
- and the best feasible solution returned by the QPU.

For a given subproblem, let:

$$z_{MWIS}^{class}$$

be the classical benchmark value, and let

$$z_{QPU}^{best}$$

be the best feasible value returned by pure quantum annealing.

A natural relative performance gap is:

$$\mathrm{Gap}_{QPU} = \frac{z_{MWIS}^{class} - z_{QPU}^{best}}{z_{MWIS}^{class}},$$

whenever $z_{MWIS}^{class} > 0$.

A small gap indicates that the annealer is reproducing the classical benchmark well on the selected reduced instance.

9.12. Execution-Time Quantities

The runtime interpretation of quantum annealing requires care. For each subproblem, the following time-related quantities should be distinguished:

- **anneal time:** the duration of one annealing run,
- **sampling time:** the aggregate time over all reads,
- **embedding time:** the time required to compute the embedding,
- **total wall-clock time:** the end-to-end time including preprocessing, embedding, execution, and decoding.

These quantities should be recorded separately because the scientific interpretation changes depending on whether the comparison is:

- solver-only,
- or full pipeline wall-clock.

9.13. Recommended Experimental Structure

For each selected subproblem Q_k, the pure annealing experiments should follow a structured procedure.

Input. A selected QUBO subproblem Q_k, its associated graph G_k, and the classical benchmark value.

Output. A set of reconstructed logical solutions, together with feasibility and objective-value statistics.

Procedure.

1. Compute an embedding of the logical QUBO onto the target hardware topology.
2. Select a set of chain-strength values to test.
3. Choose the annealing schedule configuration.
4. For each parameter configuration:
    - execute the QUBO on the QPU,
    - collect repeated samples,
    - reconstruct logical solutions,
    - check feasibility,
    - evaluate objective values.
5. Compare the best feasible QPU solution against the classical MWIS benchmark.

This procedure is specific enough to be implemented directly once the selected subproblems and QUBO models are available.

9.14. Quantities to Record

For each executed subproblem, the following quantities should be recorded:

- embedding success or failure,
- number of physical qubits used,
- average and maximum chain length,
- tested chain-strength values,
- annealing schedule configuration,
- number of reads,
- best feasible objective value z_{QPU}^{best},
- average feasible objective value,
- fraction of feasible returned samples,
- classical benchmark value z_{MWIS}^{class},
- relative gap Gap_{QPU},
- anneal time,
- total wall-clock time.

These quantities will later make it possible to identify which graph/QUBO structures are the most promising for pure annealing and where hybrid methods may become necessary.

9.15. Interpretation of the Pure Annealing Stage

The pure quantum annealing stage should be interpreted carefully. Its goal is not necessarily to outperform the classical solver immediately on every instance. Instead, it serves to answer more specific questions, such as:

- Which reduced subproblems can be embedded successfully?
- Which parameter settings produce the most reliable solutions?
- How close can pure annealing come to the classical MWIS benchmark on representative reduced instances?
- Which graph structures appear most favorable for later hybrid treatment?

Thus, Step 9 is both an execution stage and an exploratory characterization stage.

9.16. Role of Step 9 in the Full Pipeline

Step 9 is the first stage in which the reduced scheduling problem is processed on quantum hardware. It therefore connects the mathematical preparation stages of the pipeline to the actual hardware experiments.

Its role in the overall workflow is:

- to characterize pure annealing on selected reduced subproblems,
- to provide the first direct quantum-vs-classical comparisons,
- and to produce the empirical insights needed for the later hybrid classical–quantum stage.

This is precisely why it should precede any broader hybrid execution strategy.

9.17. Result of Step 9

At the end of this stage, each selected reduced QUBO instance has been embedded, executed, decoded, and benchmarked against the corresponding classical MWIS solution.

In words, the outcome of Step 9 can be summarized as follows:

> Execute the selected reduced QUBO subproblems on the quantum annealer, reconstruct the logical binary solutions, evaluate their feasibility and objective value, and compare the resulting best feasible solutions with the classical MWIS benchmark in order to characterize the performance of pure quantum annealing.

Final Interpretation of Step 9

Step 9 is the pure quantum annealing stage of the pipeline. Its function is to study how the reduced MWIS/QUBO instances behave on actual annealing hardware, and to establish the first direct comparison between QPU-generated solutions and the classical graph-optimization benchmark.

10. Step 10: Add a Hybrid Quantum-Classical Layer

After understanding pure annealing behavior, move to a hybrid architecture.

A practical structure is:

1. Classical discretization
2. Classical pruning
3. Conflict graph construction
4. Subproblem extraction
5. Quantum solve on selected dense cores
6. Classical merge and repair

10.1. 10.1 Classical Repair

After the quantum stage, use classical methods to:

- Remove residual inconsistencies
- Merge multiple subproblem solutions
- Reimpose global constraints
- Improve the final schedule locally

11. Step 11: Benchmark the Full Pipeline

A serious evaluation must compare multiple pipelines.
Suggested comparisons:

- No pruning + classical MWIS
- Pruning + classical MWIS
- Pruning + QUBO + pure annealing
- Pruning + QUBO + hybrid solver
- Small-instance richer classical benchmark

11.1. 11.1 Metrics

Measure at least:

- Objective value
- Optimality gap against best known classical solution
- Runtime
- Reduction ratio
- QUBO size reduction ratio
- Feasibility rate
- Robustness under perturbations

11.2. 11.2 Stress-Test Dimensions

Vary:

- Number of satellites
- Number of antennas
- Horizon length
- Time discretization granularity
- Conflict density
- Priority distribution
- Disturbance scenarios

12. Step 12: Study Dynamic Updates and Re-Optimization

Once the base pipeline works, study what happens when the instance changes.

Relevant dynamic events include:

- Antenna failure
- New urgent request
- Mission priority changes
- Lost visibility window

For each disturbance, study:

- How much of the graph changes
- Whether pruning can be reused
- Whether local repair is sufficient
- Whether reverse annealing helps

This is especially relevant because the scheduling problem is operational and not purely static.

13. Step 13: Keep the Core Research Questions Clear

The project should be organized around a few central research questions:

1. Can the important scheduling constraints be represented well enough as a pruned conflict graph?
2. How much can classical pruning reduce the candidate-space size?
3. After pruning, what subproblem sizes and densities remain QPU-feasible?
4. Does annealing help on the hardest reduced subgraphs compared with classical baselines?
5. Does the full hybrid pipeline outperform simpler alternatives in any meaningful regime?

14. Step 14: Suggested Implementation Order

A practical implementation order is:

Stage A

- Define the data structures
- Generate candidate slots
- Assign slot weights
- Implement basic safe pruning

Stage B

- Build the conflict graph
- Solve MWIS classically
- Validate correctness on toy cases

Stage C

- Add stronger pruning rules
- Measure graph reduction
- Compare with a richer small-instance benchmark

Stage D

- Build the QUBO
- Run classical QUBO sanity checks
- Extract small dense subgraphs

Stage E

- Run pure annealing experiments
- Tune chain strength and annealing controls
- Compare against classical MWIS on the same reduced instances

Stage F

- Add hybrid merge and repair
- Test realistic and disturbed scenarios

15. Step 15: Expected Contribution

A strong final contribution could be formulated as follows:

> We propose a hybrid pipeline for ESA ground-station scheduling in which classical pruning reduces the candidate-slot space before conflict-graph construction, enabling a compact MWIS/QUBO representation suitable for quantum annealing on selected dense subproblems.

The contribution would then be evaluated in terms of:

- Graph-size reduction
- Fidelity of the MWIS abstraction
- QPU-feasible subproblem identification
- Classical vs quantum vs hybrid performance

16. Step 16: Final Practical Advice

Do not start from the quantum stage.

Start from:

1. Candidate-slot generation
2. Classical pruning
3. Conflict graph construction
4. Classical MWIS validation

Only after those stages are working properly should the QUBO and annealing stages be introduced.

If the classical reduced-graph pipeline is not solid, then the quantum stage will not be meaningful.

Final Recommendation

The most realistic and technically sound approach is:

Classical discretization $\rightarrow$ classical pruning $\rightarrow$ conflict graph $\rightarrow$ MWIS/QUBO $\rightarrow$ quantum annealing

This is likely the strongest research direction because it respects both the complexity of the real scheduling problem and the limitations of current quantum annealing hardware.